Customer Data Analysis for Business Insights

Python Mini Project

Name: Tanmay Mandal, **Course:** Python **Date:** 22/02/2026

1. Introduction

This project analyzes customer transaction data to understand purchasing behavior, identify high-value customers, and perform RFM-based segmentation to support business decision-making.

2. Business Objective Section

The objective of this analysis is to:

- Understand customer purchasing patterns
- Identify high-value and at-risk customers
- Apply RFM analysis for segmentation
- Provide strategic business recommendations

3. Data Description

The project uses two datasets:

Customer Master Data

Contains demographic information:

Customer Transactions Data

Contains purchase details:

In [1]:
```
import os
os.getcwd()
```

Out[1]:
```
'C:\\Users\\VORTEX\\OneDrive\\Desktop\\Python Vortex\\Python Mini Project'
```

In [2]:
```
os.listdir()
```

Out[2]:
```
['.ipynb_checkpoints',
 'Customer_Data_Analysis(Final).ipynb',
 'Customer_Master_Data.csv',
 'Customer_Master_Data.xlsx',
 'Customer_Transactions.csv',
 'Final_RFM_Table.csv',
 'Final_Segmented_Customers.csv',
 'proj.ipynb',
 'project02_Customer_Data_Analysis.ipynb',
 'Rules',
 'Untitled.ipynb']
```

In [3]:
```
# Importing Important libraries
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
```

In [4]:
```
# Loading Datasets
customers = pd.read_csv("Customer_Master_Data.csv")
transactions = pd.read_csv("Customer_Transactions.csv")

print("Loaded Successfully")
```

```
Loaded Successfully
```

In [5]:
```
customers["CustomerID"] = customers["CustomerID"].astype("category")
transactions["CustomerID"] = transactions["CustomerID"].astype("category")
```

In [6]:
```
transactions.info(memory_usage="deep")
customers.info(memory_usage="deep")
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 23050 entries, 0 to 23049
Data columns (total 3 columns):
 #   Column             Non-Null Count  Dtype
---  ------             --------------  -----
 0   CustomerID         23050 non-null  category
 1   TransactionDate    23050 non-null  object
 2   TransactionAmount  23050 non-null  float64
dtypes: category(1), float64(1), object(1)
memory usage: 1.5 MB
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1000 entries, 0 to 999
Data columns (total 9 columns):
 #   Column         Non-Null Count  Dtype
---  ------         --------------  -----
 0   CustomerID     1000 non-null   category
 1   Name           1000 non-null   object
 2   Email          1000 non-null   object
 3   Gender         1000 non-null   object
 4   Age            1000 non-null   int64
 5   City           1000 non-null   object
 6   MaritalStatus  1000 non-null   object
 7   NumChildren    1000 non-null   int64
 8   JoinDate       1000 non-null   object
dtypes: category(1), int64(2), object(6)
memory usage: 456.9 KB
```

In [7]:
```
if "TransactionType" in transactions.columns:
    transactions["TransactionType"] = transactions["TransactionType"].astype("ca
```

In [8]:
```
transactions["TransactionAmount"] = pd.to_numeric(
    transactions["TransactionAmount"],
    downcast="float"
)
```

In [9]:
```
transactions.groupby(
    "CustomerID",
    sort=False,
    observed=True
)["TransactionAmount"].sum()
```

Out[9]:
```
CustomerID
CUST10771    26455.820312
CUST10100    22368.019531
CUST10031    25724.970703
CUST10987    25293.300781
CUST10831    36714.390625
                 ...
CUST10649    33119.410156
CUST10597    19349.460938
CUST10588    17384.759766
CUST10729     5052.689941
CUST10548    19975.699219
Name: TransactionAmount, Length: 1000, dtype: float32
```

In [10]:
```
customers.shape
```

Out[10]:
```
(1000, 9)
```

In [11]:
```
customers.head()
```

Out[11]:

	CustomerID	Name	Email	Gender	Age	City	MaritalStatus
0	CUST10000	Onkar Bhargava	pkeer@yahoo.com	Male	54	Delhi	Divorced
1	CUST10001	Divit Kohli	mkalita@sarin.com	Female	48	Kolkata	Married
2	CUST10002	Kiara Behl	apteanay@hotmail.com	Male	75	Kolkata	Widowed
3	CUST10003	Vaibhav Sankar	bseshadri@choudhry.info	Male	62	Pune	Divorced
4	CUST10004	Shray D'Alia	bdhillon@toor-mall.com	Male	55	Delhi	Divorced

In [12]:
```
customers.describe()
```

Out[12]:

	Age	NumChildren
count	1000.000000	1000.000000
mean	46.471000	1.311000
std	16.582525	1.190671
min	18.000000	0.000000
25%	32.000000	0.000000
50%	46.000000	1.000000
75%	60.000000	2.000000
max	75.000000	4.000000

In [13]:
```
transactions.shape
```

Out[13]: (23050, 3)

In [14]:
```
transactions.head()
```

Out[14]:

	CustomerID	TransactionDate	TransactionAmount
0	CUST10771	7/31/23	2383.070068
1	CUST10100	3/10/24	497.540009
2	CUST10031	2/17/25	536.780029
3	CUST10987	7/17/23	314.890015
4	CUST10831	12/15/24	2543.189941

In [15]:
```
transactions.describe()
```

Out[15]:

	TransactionAmount
count	23050.000000
mean	1000.138855
std	703.357483
min	3.870000
25%	482.107506
50%	844.755005
75%	1346.795044
max	6890.189941

Interpretation

The summary statistics provide insights into customer age distribution and transaction behavior.

The transaction amount shows variability, indicating the presence of both low-value and high-value purchases. This suggests customer spending behavior differs significantly across segments.

4. Data Cleaning & Preprocessing

In this section, I have checked for missing values, converted date columns, and prepared the dataset for analysis.

In [16]:
```
# Checking for Missing Values
customers.isnull().sum()
```

Out[16]:
```
CustomerID       0
Name             0
Email            0
Gender           0
Age              0
City             0
MaritalStatus    0
NumChildren      0
JoinDate         0
dtype: int64
```

In [17]:
```
transactions.isnull().sum()
```

Out[17]:
```
CustomerID           0
TransactionDate      0
TransactionAmount    0
dtype: int64
```

In [18]:
```
import warnings
warnings.filterwarnings("ignore", category=UserWarning)
```

In [19]:
```
# Converting Date Columns to Datetime
customers['JoinDate'] = pd.to_datetime(customers['JoinDate'])
transactions['TransactionDate'] = pd.to_datetime(transactions['TransactionDate']
```

In [20]:
```
customers.info()
transactions.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1000 entries, 0 to 999
Data columns (total 9 columns):
 #   Column         Non-Null Count  Dtype
---  ------         --------------  -----
 0   CustomerID     1000 non-null   category
 1   Name           1000 non-null   object
 2   Email          1000 non-null   object
 3   Gender         1000 non-null   object
 4   Age            1000 non-null   int64
 5   City           1000 non-null   object
 6   MaritalStatus  1000 non-null   object
 7   NumChildren    1000 non-null   int64
 8   JoinDate       1000 non-null   datetime64[ns]
dtypes: category(1), datetime64[ns](1), int64(2), object(5)
memory usage: 104.7+ KB
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 23050 entries, 0 to 23049
Data columns (total 3 columns):
 #   Column             Non-Null Count  Dtype
---  ------             --------------  -----
 0   CustomerID         23050 non-null  category
 1   TransactionDate    23050 non-null  datetime64[ns]
 2   TransactionAmount  23050 non-null  float32
dtypes: category(1), datetime64[ns](1), float32(1)
memory usage: 355.4 KB
```

In [21]:
```
# Checking for Duplicate Customers
customers.duplicated(subset='CustomerID').sum()
```

Out[21]:
```
np.int64(0)
```

In [22]:
```
# Validate CustomerIDs in Transactions
invalid_ids = transactions[~transactions['CustomerID'].isin(customers['CustomerI
invalid_ids.shape
```

Out[22]:
```
(0, 3)
```

In [23]:
```
# Removing Negative or Zero Transaction Amounts
transactions[transactions['TransactionAmount'] <= 0].shape
```

Out[23]:
```
(0, 3)
```

In [24]:
```
# Detecting Outliers Using the IQR Method
# for extreme value
Q1 = transactions['TransactionAmount'].quantile(0.25)
Q3 = transactions['TransactionAmount'].quantile(0.75)

IQR = Q3 - Q1

lower_bound = Q1 - 1.5 * IQR
upper_bound = Q3 + 1.5 * IQR

lower_bound, upper_bound
```

Out[24]:
```
(np.float64(-814.9238014221191), np.float64(2643.8263511657715))
```

In [25]:
```
# for outliners
outliers = transactions[
```

```
    (transactions['TransactionAmount'] < lower_bound) |
    (transactions['TransactionAmount'] > upper_bound)
]

outliers.shape
```

Out[25]:
```
(736, 3)
```

In [26]:
```
# Standardize Text Columns
customers['Gender'] = customers['Gender'].str.strip().str.title()
customers['City'] = customers['City'].str.strip().str.title()
customers['MaritalStatus'] = customers['MaritalStatus'].str.strip().str.title()
```

In [27]:
```
customers['Gender'].unique()
customers['MaritalStatus'].unique()
```

Out[27]:
```
array(['Divorced', 'Married', 'Widowed', 'Single'], dtype=object)
```

Interpretation

No duplicate records were found, ensuring data reliability and preventing inflated revenue calculations.

5. Exploratory Data Analysis (EDA)

This section analyzes transaction patterns, revenue trends, and customer distribution.

Customer Master Dataset

In [28]:
```
customers['CustomerID'].nunique() # Total Unique Customers
```

Out[28]:
```
1000
```

In [29]:
```
customers['Gender'].value_counts() #Gender Distribution
```

Out[29]:
```
Gender
Female          340
Male            332
Not Disclosed   328
Name: count, dtype: int64
```

In [30]:
```
(customers['Gender'].value_counts(normalize=True) * 100).round(2)
```

Out[30]:
```
Gender
Female          34.0
Male            33.2
Not Disclosed   32.8
Name: proportion, dtype: float64
```

In [31]:
```
customers['Age'].mean()
```

Out[31]:
```
np.float64(46.471)
```

In [32]:
```
# Top 5 Cities by Customer Count
customers['City'].value_counts().head(5)
```

Out[32]:
```
City
Lucknow      110
Jaipur       109
Hyderabad    104
Ahmedabad    102
Bangalore    102
Name: count, dtype: int64
```

In [33]:
```
# Married vs Single Percentage
customers['MaritalStatus'].value_counts(normalize=True) * 100
```

Out[33]:
```
MaritalStatus
Widowed     28.0
Divorced    25.0
Single      24.6
Married     22.4
Name: proportion, dtype: float64
```

In [34]:
```
# Creating Age Groups
bins = [18, 25, 35, 45, 55, 65, 100]
labels = ['18-25', '26-35', '36-45', '46-55', '56-65', '65+']

customers['AgeGroup'] = pd.cut(customers['Age'], bins=bins, labels=labels)

customers['AgeGroup'].value_counts()
```

Out[34]:
```
AgeGroup
36-45    190
26-35    182
65+      180
46-55    175
56-65    149
18-25    112
Name: count, dtype: int64
```

Transaction Dataset

In [35]:
```
#  Total Transactions
transactions.shape[0]
```

Out[35]:
```
23050
```

In [36]:
```
#  Total Revenue
transactions['TransactionAmount'].sum()
```

Out[36]:
```
np.float32(2.30532e+07)
```

In [37]:
```
#  Average Transaction Amount
transactions['TransactionAmount'].mean()
```

Out[37]:
```
np.float32(1000.13885)
```

In [38]:
```
#  Highest & Lowest Transactions
transactions['TransactionAmount'].max()
```

Out[38]: 6890.18994140625

In [39]:
```
transactions['TransactionAmount'].min()
```

Out[39]: 3.869999885559082

In [40]:
```
#  Monthly Revenue Trend
transactions['YearMonth'] = transactions['TransactionDate'].dt.to_period('M')

monthly_revenue = transactions.groupby('YearMonth')['TransactionAmount'].sum()

monthly_revenue
```

Out[40]:

YearMonth	
2020-08	4920.700195
2020-09	8087.860352
2020-10	14035.639648
2020-11	26427.519531
2020-12	44923.808594
2021-01	46313.328125
2021-02	56751.058594
2021-03	73427.828125
2021-04	84110.148438
2021-05	83429.023438
2021-06	97844.992188
2021-07	106881.601562
2021-08	118871.710938
2021-09	118180.617188
2021-10	146381.546875
2021-11	131805.343750
2021-12	155506.953125
2022-01	174315.593750
2022-02	167072.437500
2022-03	223836.843750
2022-04	236575.468750
2022-05	237187.921875
2022-06	234176.703125
2022-07	251243.125000
2022-08	245218.093750
2022-09	269509.718750
2022-10	281834.593750
2022-11	291384.000000
2022-12	312764.312500
2023-01	346707.656250
2023-02	318612.156250
2023-03	376909.625000
2023-04	382688.937500
2023-05	412299.937500
2023-06	395256.812500
2023-07	419527.656250
2023-08	423156.187500
2023-09	549953.750000
2023-10	492321.968750
2023-11	516104.875000
2023-12	556071.250000
2024-01	628087.812500
2024-02	562946.500000
2024-03	598707.375000
2024-04	619173.062500
2024-05	746989.750000
2024-06	727259.937500
2024-07	782661.250000
2024-08	795313.000000
2024-09	737544.500000
2024-10	777659.687500
2024-11	720391.375000
2024-12	741476.437500
2025-01	798789.125000
2025-02	693491.875000
2025-03	752082.062500
2025-04	747412.187500
2025-05	770017.375000
2025-06	721939.687500

```
2025-07    698627.312500
Freq: M, Name: TransactionAmount, dtype: float32
```

In [41]:
```
#  Top 10 Revenue-Generating Days
daily_revenue = transactions.groupby('TransactionDate')['TransactionAmount'].sum

daily_revenue.sort_values(ascending=False).head(10)
```

Out[41]:
```
TransactionDate
2024-08-30    50618.019531
2024-08-11    47048.371094
2024-10-31    44940.781250
2024-10-26    44171.261719
2024-09-01    42801.308594
2024-05-25    42017.839844
2024-11-06    41771.019531
2024-06-12    41352.539062
2024-01-06    39692.089844
2024-05-15    38925.570312
Name: TransactionAmount, dtype: float32
```

Merging Customers & Transactions

In [42]:
```
merged_data = pd.merge(
    customers,
    transactions,
    on='CustomerID',
    how='left'
)

merged_data.head()
```

Out[42]:

	CustomerID	Name	Email	Gender	Age	City	MaritalStatus	NumCh
0	CUST10000	Onkar Bhargava	pkeer@yahoo.com	Male	54	Delhi	Divorced	
1	CUST10000	Onkar Bhargava	pkeer@yahoo.com	Male	54	Delhi	Divorced	
2	CUST10000	Onkar Bhargava	pkeer@yahoo.com	Male	54	Delhi	Divorced	
3	CUST10000	Onkar Bhargava	pkeer@yahoo.com	Male	54	Delhi	Divorced	
4	CUST10000	Onkar Bhargava	pkeer@yahoo.com	Male	54	Delhi	Divorced	

In [43]:
```
merged_data.shape
```

Out[43]:
```
(23050, 13)
```

In [44]:
```
merged_data['TransactionAmount'].isnull().sum()
```

Out[44]:
```
np.int64(0)
```

Total Spending per Customer

In [45]:
```
total_spending = merged_data.groupby('CustomerID', observed=True)['TransactionAm

total_spending.head()
```

Out[45]:
```
CustomerID
CUST10000    21265.490234
CUST10001    28654.308594
CUST10002    23884.029297
CUST10003    24206.029297
CUST10004    25565.300781
Name: TransactionAmount, dtype: float32
```

In [46]:
```
total_spending = (
    merged_data
    .groupby('CustomerID', observed=True)['TransactionAmount']
    .sum()
)
```

In [47]:
```
total_spending.describe()
```

Out[47]:
```
count     1000.000000
mean     23053.199219
std       5622.441895
min       5052.689941
25%      18965.462891
50%      22969.820312
75%      26827.391602
max      44784.992188
Name: TransactionAmount, dtype: float64
```

Total Transactions per Customer

In [48]:
```
total_transactions = merged_data.groupby('CustomerID', observed=True)['Transacti

total_transactions.head()
```

Out[48]:
```
CustomerID
CUST10000    23
CUST10001    30
CUST10002    24
CUST10003    25
CUST10004    19
Name: TransactionAmount, dtype: int64
```

In [49]:
```
total_transactions = merged_data.groupby('CustomerID', observed=True)['Transacti

total_transactions.head()
```

Out[49]:
```
CustomerID
CUST10000    23
CUST10001    30
CUST10002    24
CUST10003    25
CUST10004    19
Name: TransactionAmount, dtype: int64
```

In [50]:
```
total_transactions.mean()
```

Out[50]:
```
np.float64(23.05)
```

Average Spending per Customer

In [51]:
```
avg_spending = total_spending / total_transactions

avg_spending.head()
```

Out[51]:
```
CustomerID
CUST10000     924.586532
CUST10001     955.143620
CUST10002     995.167887
CUST10003     968.241172
CUST10004    1345.542146
Name: TransactionAmount, dtype: float64
```

In [52]:
```
avg_spending.describe()
```

Out[52]:
```
count    1000.000000
mean     1002.637318
std       146.092813
min       625.343825
25%       901.710593
50%       995.191247
75%      1093.642017
max      1658.542969
Name: TransactionAmount, dtype: float64
```

6. RFM Analysis

RFM (Recency, Frequency, Monetary) analysis is used to evaluate customer value and segment customers based on purchasing behavior.

In [53]:
```
# Defining Reference Date
merged_data['TransactionDate'] = pd.to_datetime(merged_data['TransactionDate'])
```

In [54]:
```
reference_date = merged_data['TransactionDate'].max() + pd.Timedelta(days=1)
reference_date
```

Out[54]:
```
Timestamp('2025-07-30 00:00:00')
```

In [55]:
```
customers['JoinDate'] = pd.to_datetime(customers['JoinDate'])
```

Customer Lifetime (Days Since Join)

In [56]:
```
customers['CustomerLifetimeDays'] = (
    reference_date - customers['JoinDate']
).dt.days

customers[['CustomerID', 'JoinDate', 'CustomerLifetimeDays']].head()
```

Out[56]:

	CustomerID	JoinDate	CustomerLifetimeDays
0	CUST10000	2021-02-22	1619
1	CUST10001	2023-12-06	602
2	CUST10002	2023-08-23	707
3	CUST10003	2022-11-17	986
4	CUST10004	2022-12-04	969

In [57]:
```
customers['CustomerLifetimeDays'].describe()
```

Out[57]:
```
count    1000.000000
mean     1101.933000
std       429.190903
min       367.000000
25%       724.500000
50%      1115.500000
75%      1478.500000
max      1827.000000
Name: CustomerLifetimeDays, dtype: float64
```

Calculating Recency(R)

In [58]:
```
recency = merged_data.groupby('CustomerID', observed=True)['TransactionDate'].ma

recency = (reference_date - recency).dt.days

recency.head()
```

Out[58]:
```
CustomerID
CUST10000    13
CUST10001    35
CUST10002    18
CUST10003    81
CUST10004     8
Name: TransactionDate, dtype: int64
```

Calculating Frequency(F)

In [59]:
```
frequency = merged_data.groupby('CustomerID', observed=True)['TransactionAmount'

frequency.head()
```

Out[59]:
```
CustomerID
CUST10000    23
CUST10001    30
CUST10002    24
CUST10003    25
CUST10004    19
Name: TransactionAmount, dtype: int64
```

Calculating Monetary(M)

In [60]:
```
monetary = merged_data.groupby('CustomerID', observed=True)['TransactionAmount']
```

```
monetary.head()
```

Out[60]:
```
CustomerID
CUST10000    21265.490234
CUST10001    28654.308594
CUST10002    23884.029297
CUST10003    24206.029297
CUST10004    25565.300781
Name: TransactionAmount, dtype: float32
```

In [61]:
```
df_rfm = pd.DataFrame({
    'Recency': recency,
    'Frequency': frequency,
    'Monetary': monetary
})

df_rfm.head()
```

Out[61]:

CustomerID	Recency	Frequency	Monetary
CUST10000	13	23	21265.490234
CUST10001	35	30	28654.308594
CUST10002	18	24	23884.029297
CUST10003	81	25	24206.029297
CUST10004	8	19	25565.300781

RFM Scoring

In [62]:
```
df_rfm['R_Score'] = pd.qcut(df_rfm['Recency'], 5, labels=[5,4,3,2,1])
df_rfm['F_Score'] = pd.qcut(df_rfm['Frequency'], 5, labels=[1,2,3,4,5])
df_rfm['M_Score'] = pd.qcut(df_rfm['Monetary'], 5, labels=[1,2,3,4,5])
```

Combined Score

In [63]:
```
df_rfm['RFM_Score'] = (
    df_rfm['R_Score'].astype(str) +
    df_rfm['F_Score'].astype(str) +
    df_rfm['M_Score'].astype(str)
)

df_rfm.head()
```

Out[63]:

CustomerID	Recency	Frequency	Monetary	R_Score	F_Score	M_Score	RFM_Score
CUST10000	13	23	21265.490234	4	3	2	432
CUST10001	35	30	28654.308594	3	5	5	355
CUST10002	18	24	23884.029297	4	3	3	433
CUST10003	81	25	24206.029297	1	4	3	143
CUST10004	8	19	25565.300781	5	1	4	514

In [64]:
```
df_rfm['RFM_Score'].value_counts().sort_values(ascending=False).head(10)
```

Out[64]:
```
RFM_Score
311    37
555    35
111    34
411    33
511    28
355    26
211    26
544    22
455    22
122    21
Name: count, dtype: int64
```

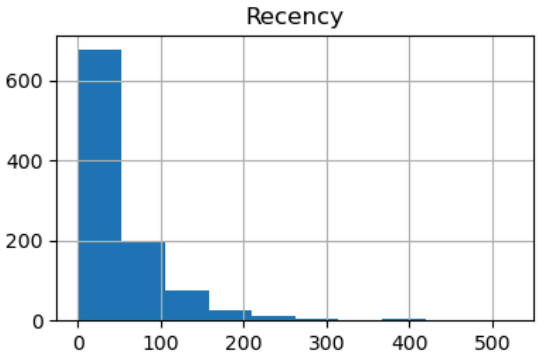
In [65]:
```
df_rfm['R_Score'] = df_rfm['R_Score'].astype(int)
df_rfm['F_Score'] = df_rfm['F_Score'].astype(int)
df_rfm['M_Score'] = df_rfm['M_Score'].astype(int)
```

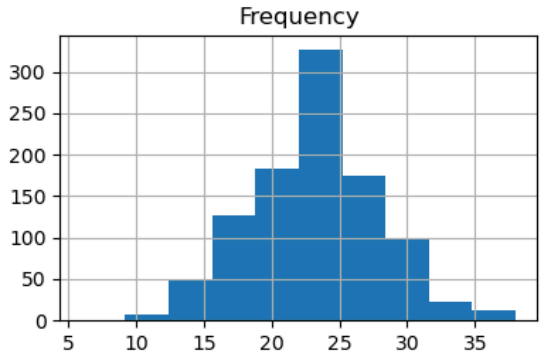
RFM analysis evaluates customer value using three dimensions:

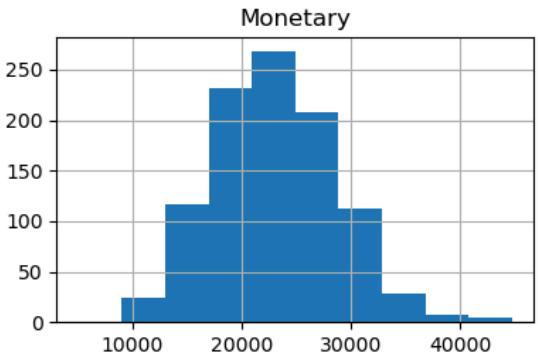
- Recency: How recently a customer made a purchase
- Frequency: How often a customer purchases
- Monetary: How much money a customer spends

Customers with low recency, high frequency, and high monetary value are considered the most valuable.

In [66]:
```
df_rfm[['Recency','Frequency', 'Monetary']].hist(figsize=(10,6))
plt.show()
```







Interpretation

The distributions indicate skewness in Monetary values, confirming that a small number of customers contribute significantly higher revenue.

7. Customer Segmentation

Customers are grouped into segments such as Champions, Loyal Customers, At Risk, Hibernating, and Potential Loyalists based on RFM score combinations.

In [67]:
```
# Creating Segment Column
def segment_customer(row):
    if row['R_Score'] == 5 and row['F_Score'] == 5 and row['M_Score'] == 5:
        return 'Champions'
    elif row['R_Score'] >= 4 and row['F_Score'] >= 4:
        return 'Loyal Customers'
    elif row['R_Score'] <= 2 and row['F_Score'] <= 2:
        return 'At Risk.'
    elif row['R_Score'] == 5:
        return 'Hibernating'
    else:
        return 'Potential Loyalist.'

df_rfm['Segment'] = df_rfm.apply(segment_customer, axis=1)

df_rfm.head()
```

Out[67]:

CustomerID	Recency	Frequency	Monetary	R_Score	F_Score	M_Score	RFM_Score
CUST10000	13	23	21265.490234	4	3	2	432
CUST10001	35	30	28654.308594	3	5	5	355
CUST10002	18	24	23884.029297	4	3	3	433
CUST10003	81	25	24206.029297	1	4	3	143
CUST10004	8	19	25565.300781	5	1	4	514

In [68]:
```
df_rfm['Segment'].value_counts()
```

Out[68]:
```
Segment
Potential Loyalist.    523
At Risk.               193
Hibernating            127
Loyal Customers        122
Champions               35
Name: count, dtype: int64
```

In [69]:
```
# Revenue per Segment
segment_revenue = df_rfm.groupby('Segment')['Monetary'].sum()

segment_revenue.sort_values(ascending=False)
```

Out[69]:
```
Segment
Potential Loyalist.    1.232387e+07
At Risk.               3.734111e+06
Loyal Customers        3.298518e+06
Hibernating            2.597312e+06
Champions              1.099394e+06
Name: Monetary, dtype: float32
```

In [70]:
```
# Percentage Revenue Contribution
(segment_revenue / segment_revenue.sum() * 100).round(2)
```

Out[70]:
```
Segment
At Risk.               16.200001
Champions               4.770000
Hibernating            11.270000
Loyal Customers        14.310000
Potential Loyalist.    53.459999
Name: Monetary, dtype: float32
```

In [71]:
```
df_rfm[df_rfm['Segment'] == 'At Risk.'].head()
```

Out[71]:

CustomerID	Recency	Frequency	Monetary	R_Score	F_Score	M_Score	RFM_Score
CUST10007	86	15	14957.060547	1	1	1	111
CUST10018	72	19	20383.919922	2	1	2	212
CUST10022	60	15	21368.650391	2	1	3	213
CUST10030	72	17	15128.229492	2	1	1	211
CUST10034	153	21	30387.640625	1	2	5	125

In [72]:
```
df_rfm[df_rfm['Segment'] == 'At Risk.'].shape
```

Out[72]:
```
(193, 8)
```

In [73]:
```
at_risk_revenue = df_rfm[df_rfm['Segment']=='At Risk.']['Monetary'].sum()
total_revenue = df_rfm['Monetary'].sum()
```

In [74]:
```
risk_percentage = (at_risk_revenue / total_revenue) * 100
risk_percentage
```

Out[74]:
```
np.float32(16.1978)
```

Revenue Risk Analysis

Approximately 16.19% of total revenue is generated by customers classified as At Risk.

This represents a significant revenue exposure. If these customers churn, the business could lose nearly one-sixth of its revenue.

Immediate re-engagement strategies such as personalized discounts, loyalty incentives, and targeted email campaigns should be implemented to retain this segment.

In [75]:
```
# Identifying Top Revenue Segment
segment_revenue.sort_values(ascending=False)
```

Out[75]:
```
Segment
Potential Loyalist.    1.232387e+07
At Risk.               3.734111e+06
Loyal Customers        3.298518e+06
Hibernating            2.597312e+06
Champions              1.099394e+06
Name: Monetary, dtype: float32
```

Interpretation

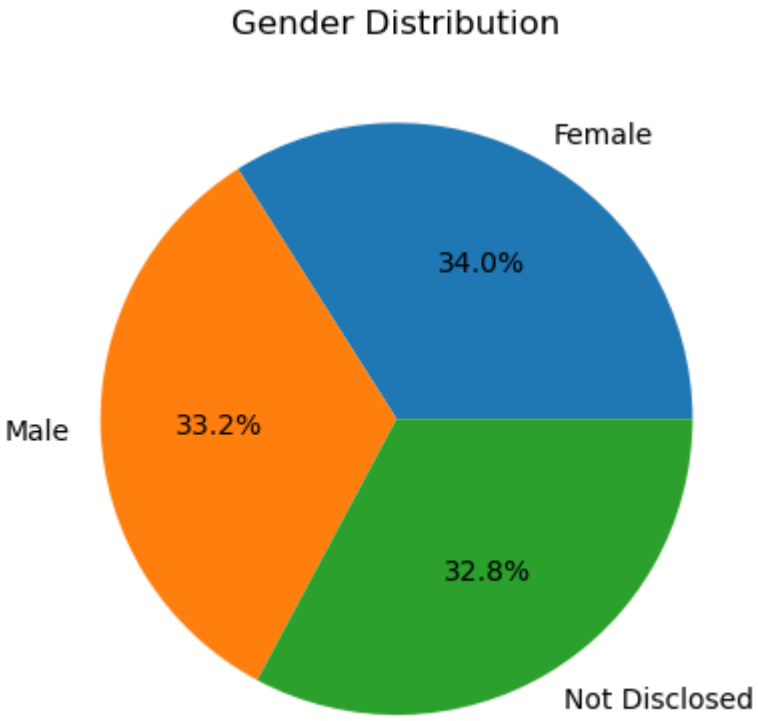
Revenue contribution percentage confirms that Champions and Loyal Customers generate the highest share of total revenue.

Retaining these customers is critical for long-term profitability.

8. Data Visualization

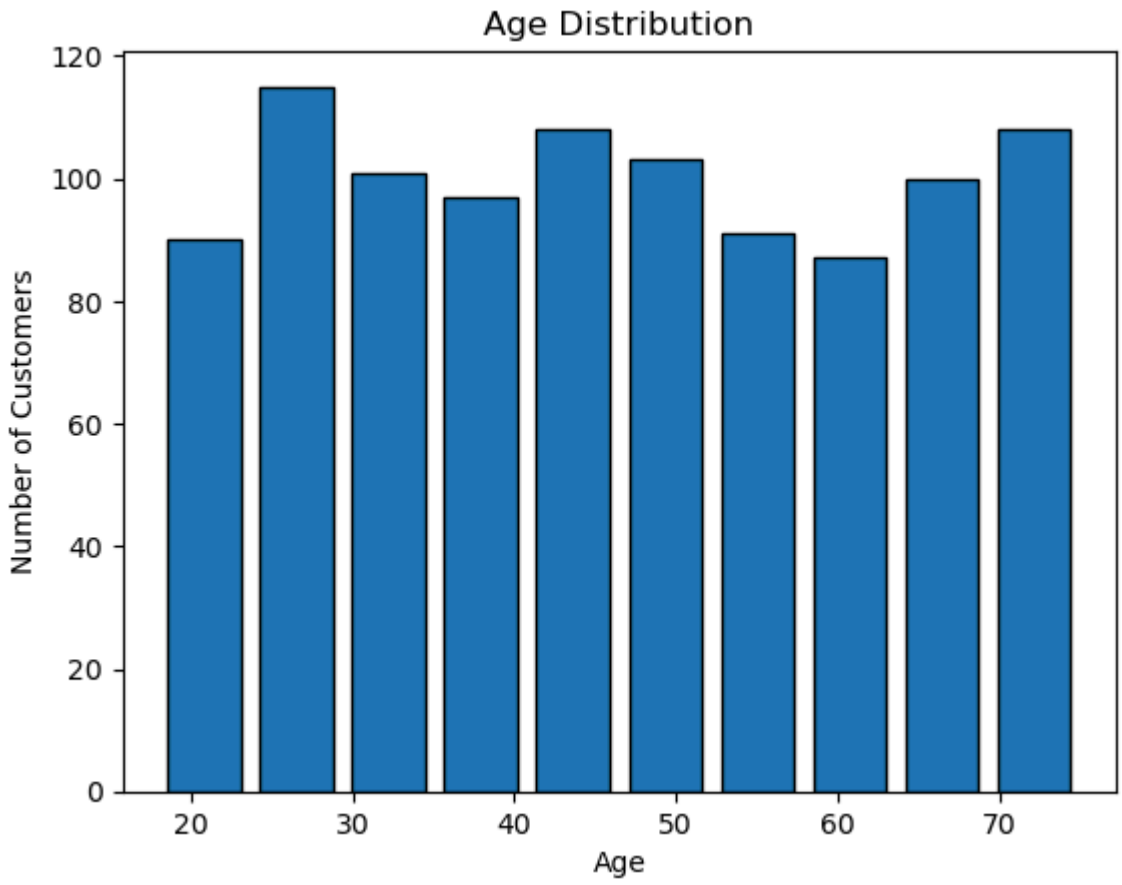
Gender Distribution (Pie Chart)

In [76]:
```
import matplotlib.pyplot as plt
# Gender Distribution (Pie Chart)
gender_counts = customers['Gender'].value_counts()

plt.figure()
plt.pie(gender_counts, labels=gender_counts.index, autopct='%1.1f%%')
plt.title('Gender Distribution')
plt.show()
```



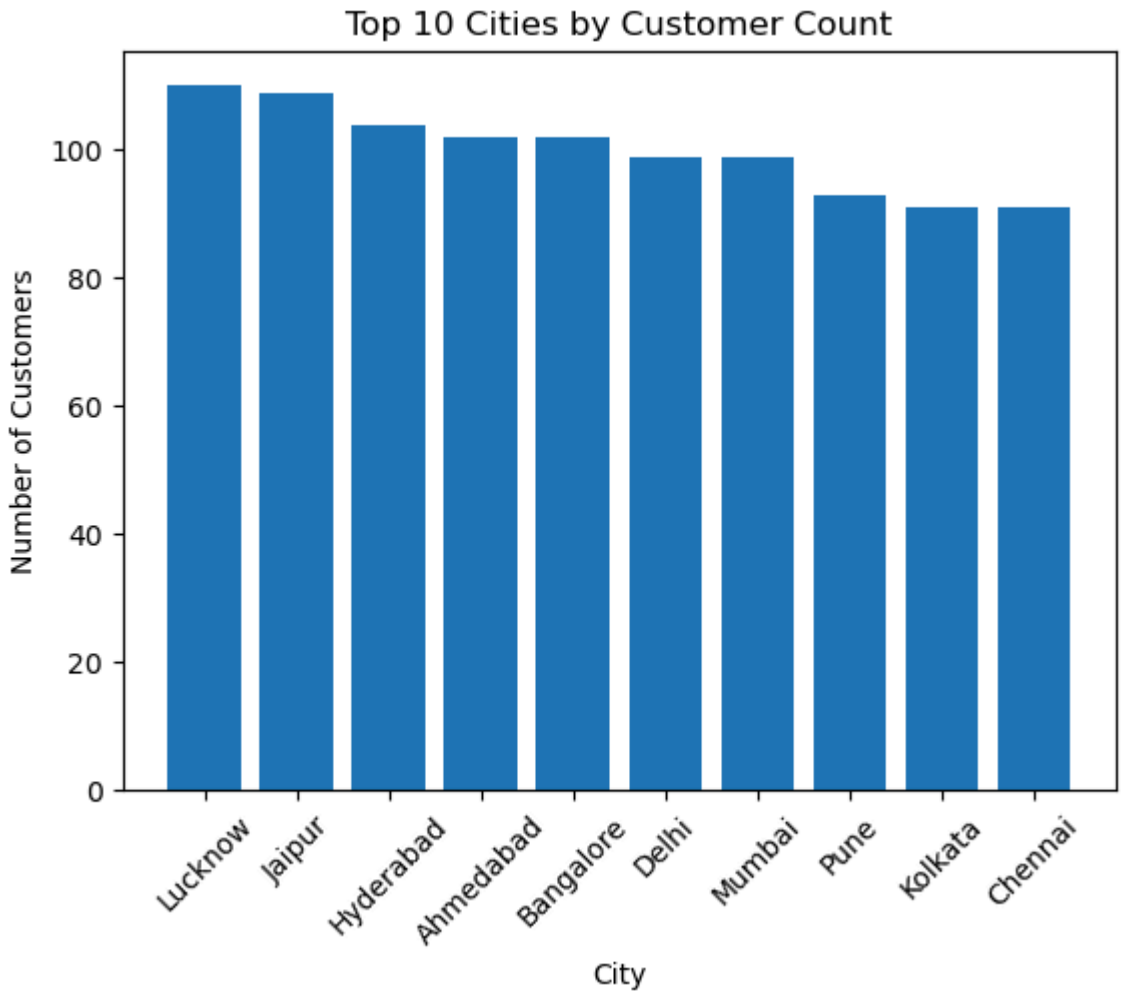
Age Distribution (Histogram)

In [77]:
```
plt.figure()
plt.hist(customers['Age'], bins=10, edgecolor='black', rwidth=0.8)
plt.title('Age Distribution')
plt.xlabel('Age')
plt.ylabel('Number of Customers')
plt.show()
```



City-wise Customer Count (Bar Chart)

In [78]:
```
city_counts = customers['City'].value_counts().head(10)

plt.figure()
plt.bar(city_counts.index, city_counts.values)
plt.title('Top 10 Cities by Customer Count')
plt.xticks(rotation=45)
plt.xlabel('City')
plt.ylabel('Number of Customers')
plt.show()
```



In [79]:
```
# checking- date is datetime
merged_data['TransactionDate'] = pd.to_datetime(merged_data['TransactionDate'])

# Creating YearMonth column
merged_data['YearMonth'] = merged_data['TransactionDate'].dt.to_period('M')

# Calculate monthly revenue
monthly_revenue = merged_data.groupby('YearMonth')['TransactionAmount'].sum()

monthly_revenue.head()
```

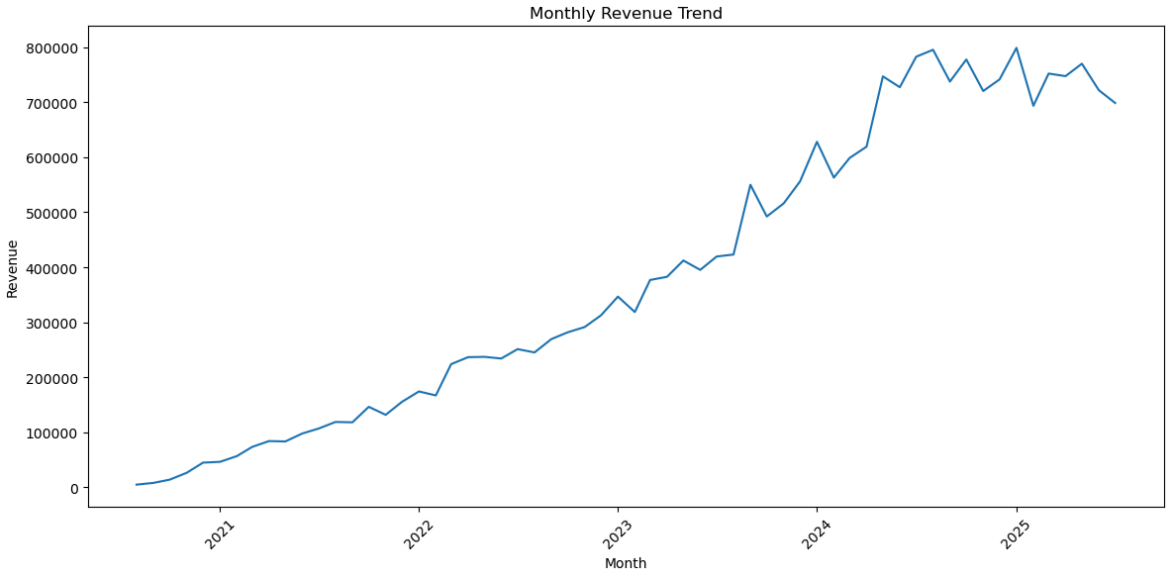
Out[79]:
```
YearMonth
2020-08     4920.700195
2020-09     8087.859863
2020-10    14035.639648
2020-11    26427.519531
2020-12    44923.808594
Freq: M, Name: TransactionAmount, dtype: float32
```

Monthly Revenue Trend (Line Chart)

In [80]:
```
monthly_revenue.index = monthly_revenue.index.to_timestamp()
```

In [81]:
```
plt.figure(figsize=(12,6))

plt.plot(monthly_revenue.index, monthly_revenue.values)

plt.title('Monthly Revenue Trend')
```

```
plt.xlabel('Month')
plt.ylabel('Revenue')

plt.xticks(rotation=45)
plt.tight_layout()
plt.show()
```



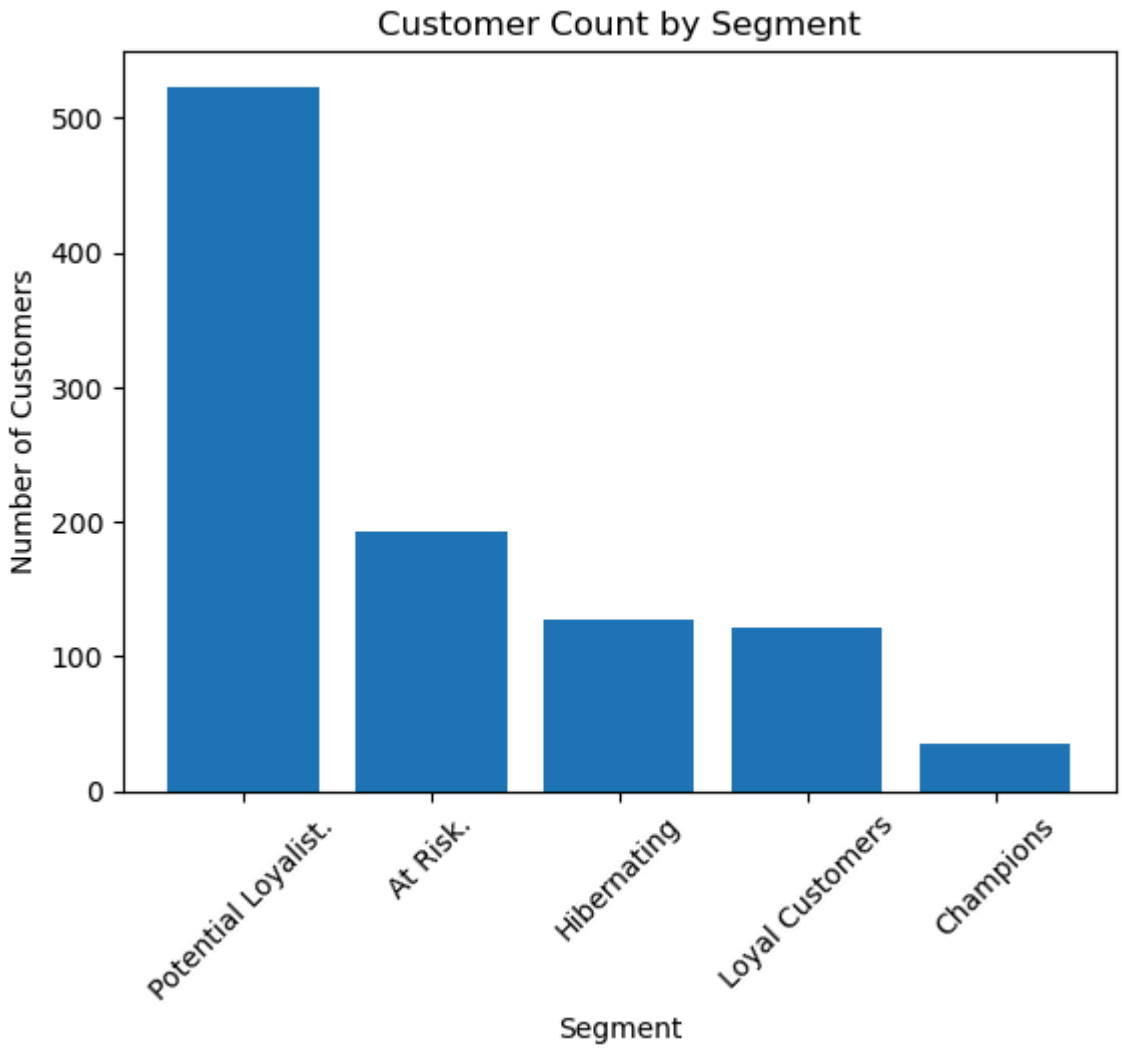
Interpretation

Revenue shows a consistent upward trend, indicating business growth and effective customer retention.

Seasonal peaks may indicate promotional campaigns or festive demand.

Segment-wise Customer Count (Bar Chart)

In [82]:
```
segment_counts = df_rfm['Segment'].value_counts()

plt.figure()
plt.bar(segment_counts.index, segment_counts.values)
plt.xticks(rotation=45)
plt.title('Customer Count by Segment')
plt.xlabel('Segment')
plt.ylabel('Number of Customers')
plt.show()
```



Interpretation

The distribution of segments shows that while a portion of customers are highly engaged, a considerable number fall into At Risk and Hibernating categories.

This indicates an opportunity for reactivation campaigns.

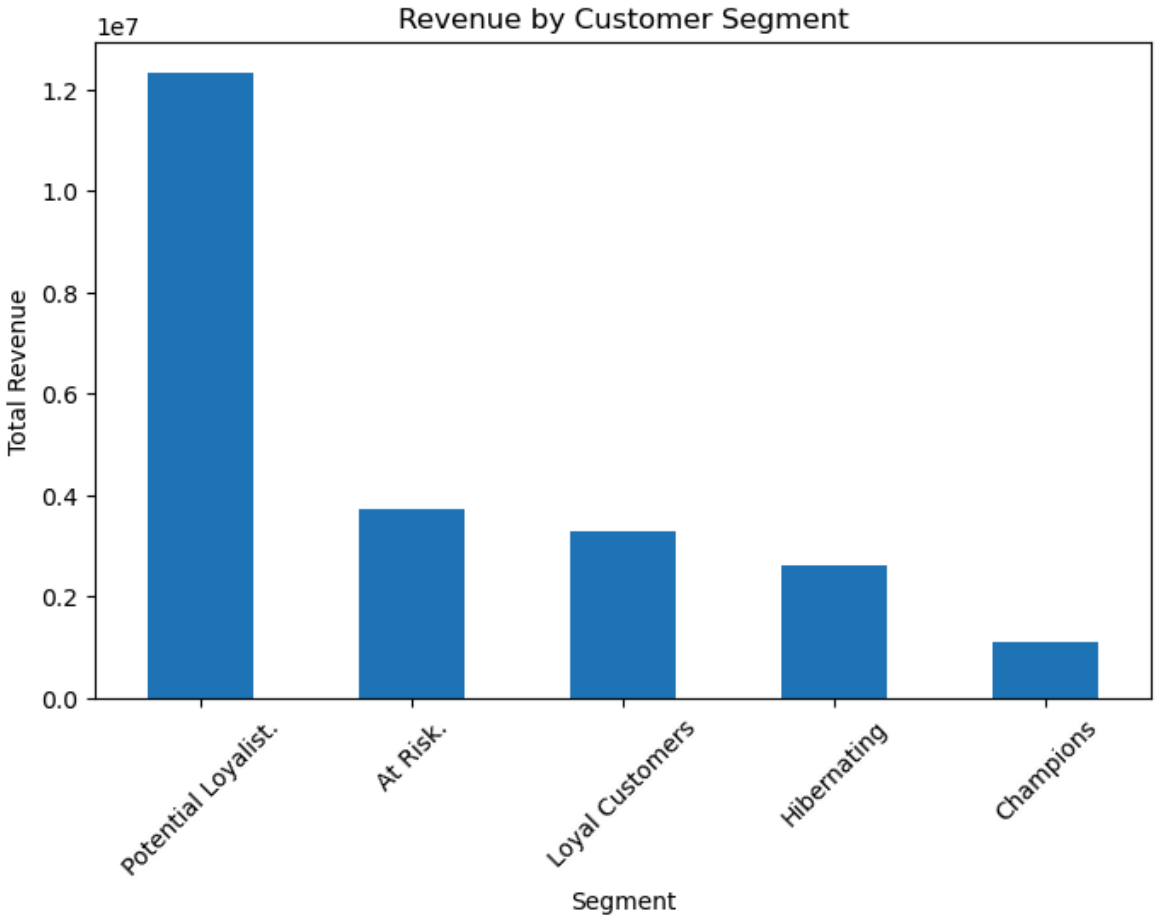
Segment-wise Revenue

In [83]:
```
segment_revenue = df_rfm.groupby("Segment")["Monetary"].sum().sort_values(ascend
segment_revenue
```

Out[83]:
```
Segment
Potential Loyalist.    1.232387e+07
At Risk.               3.734111e+06
Loyal Customers        3.298518e+06
Hibernating            2.597312e+06
Champions              1.099394e+06
Name: Monetary, dtype: float32
```

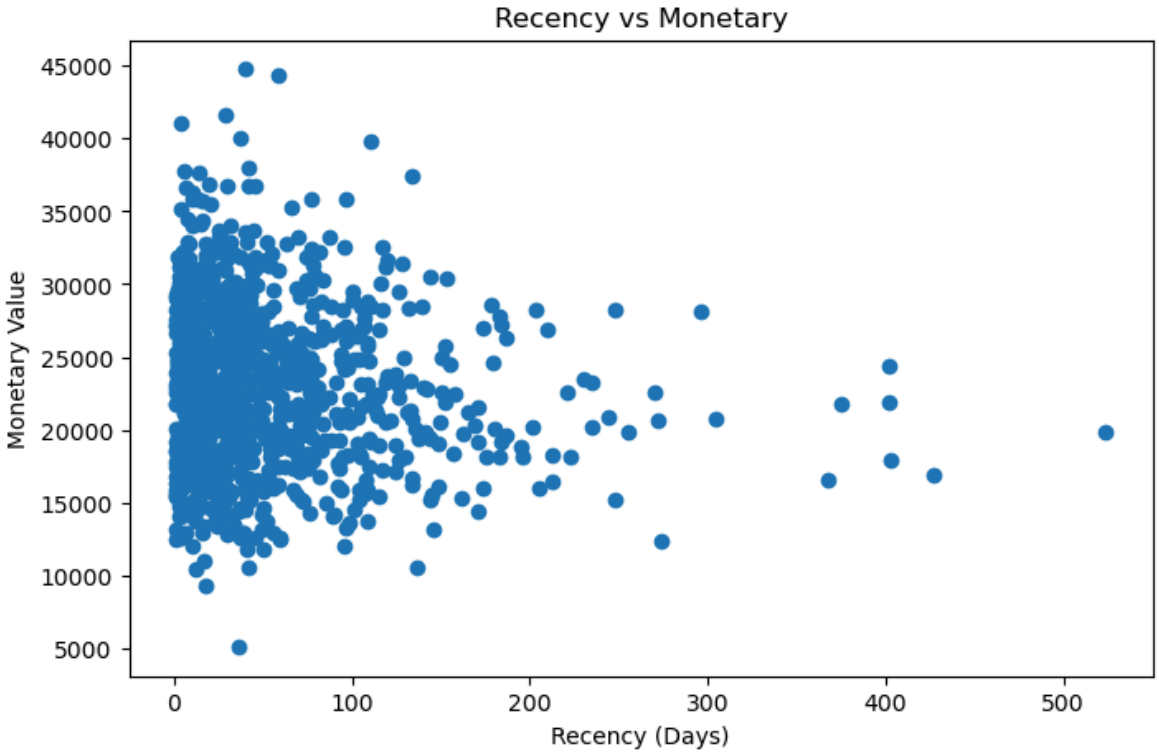
In [84]:
```
import matplotlib.pyplot as plt

segment_revenue.plot(kind="bar", figsize=(8,5))
plt.title("Revenue by Customer Segment")
plt.xlabel("Segment")
```

```
plt.ylabel("Total Revenue")
plt.xticks(rotation=45)
plt.show()
```



Scatter Plot: Recency vs Monetary

In [85]:
```
plt.figure(figsize=(8,5))
plt.scatter(df_rfm["Recency"], df_rfm["Monetary"])
plt.title("Recency vs Monetary")
plt.xlabel("Recency (Days)")
plt.ylabel("Monetary Value")
plt.show()
```



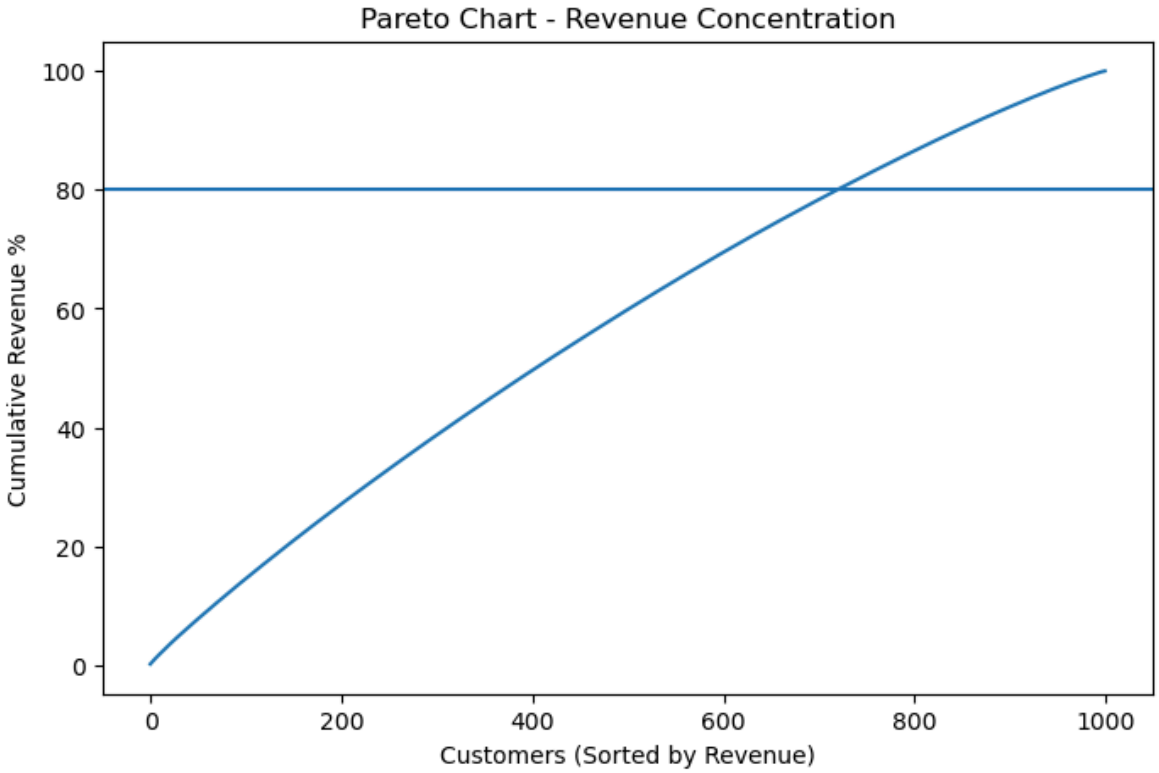
Pareto Chart (Top 20% Customers Revenue Share)

In [86]:
```
df_rfm_sorted = df_rfm.sort_values("Monetary", ascending=False)
```

In [87]:
```
df_rfm_sorted = df_rfm.sort_values(by="Monetary", ascending=False)
df_rfm_sorted["CumulativeRevenue"] = df_rfm_sorted["Monetary"].cumsum()
df_rfm_sorted["CumulativePercent"] = (
    df_rfm_sorted["CumulativeRevenue"] / df_rfm_sorted["Monetary"].sum()
) * 100
```

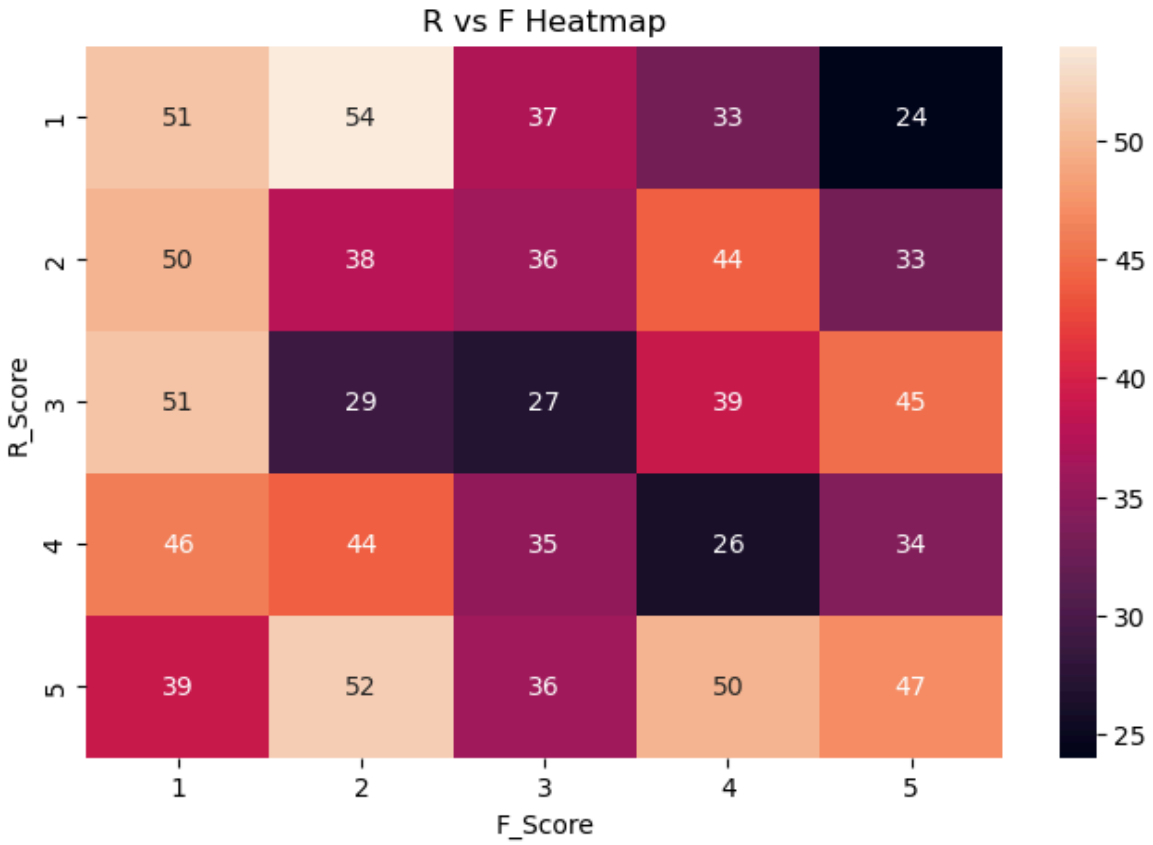
In [88]:
```
df_rfm_sorted["CumulativeRevenue"] = df_rfm_sorted["Monetary"].cumsum()
df_rfm_sorted["CumulativePercent"] = 100 * df_rfm_sorted["CumulativeRevenue"] /
```

In [89]:
```
import matplotlib.pyplot as plt
plt.figure(figsize=(8,5))
plt.plot(df_rfm_sorted["CumulativePercent"].values)
plt.axhline(80)
plt.title("Pareto Chart - Revenue Concentration")
plt.xlabel("Customers (Sorted by Revenue)")
plt.ylabel("Cumulative Revenue %")
plt.show()
```



Heatmap of R vs F Scores

In [90]:
```
import seaborn as sns
import matplotlib.pyplot as plt

df_rfm_pivot = df_rfm.pivot_table(
    index="R_Score",
    columns="F_Score",
    values="Monetary",
    aggfunc="count"
).fillna(0)

plt.figure(figsize=(8,5))
sns.heatmap(df_rfm_pivot, annot=True, fmt=".0f")
plt.title("R vs F Heatmap")
plt.show()
```



Interpretation

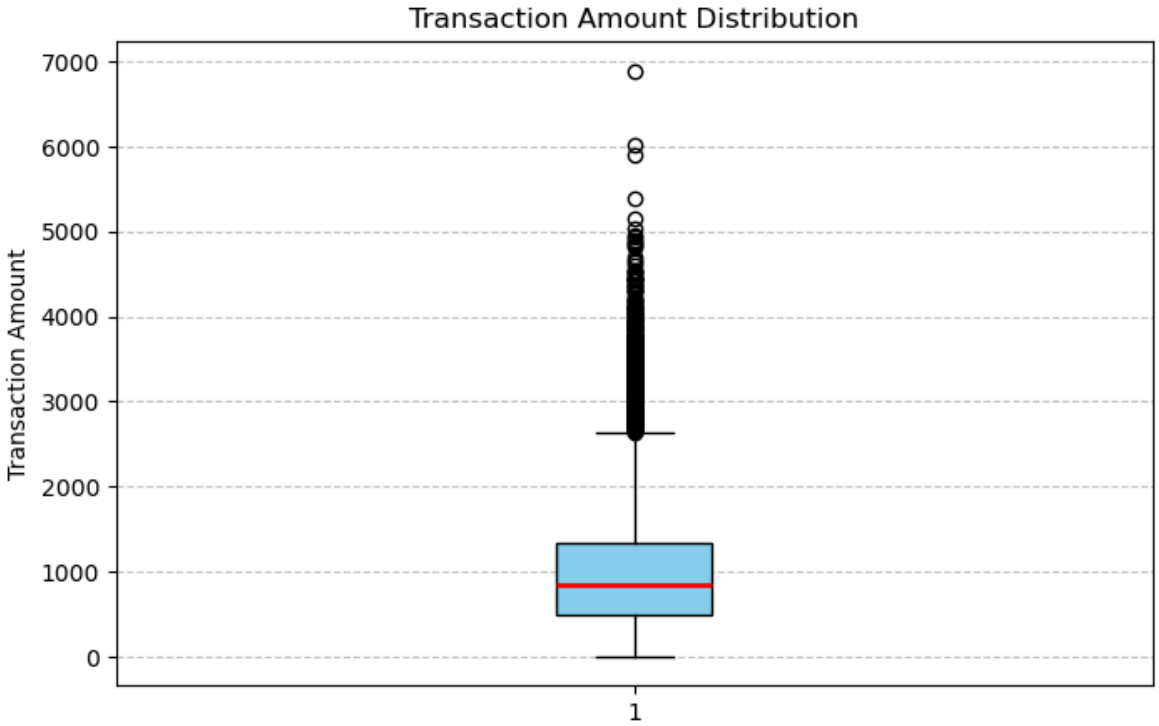
Frequency and Monetary show a positive correlation, indicating that customers who purchase more frequently tend to spend more overall.

Recency shows negative correlation with Frequency, meaning customers who purchased recently are generally more active buyers.

This suggests that maintaining purchase frequency directly impacts revenue growth.

Box plotting of Transaction Amount Distribution

In [91]:
```
plt.figure(figsize=(8,5))

plt.boxplot(
    transactions["TransactionAmount"],
    patch_artist=True,
    boxprops=dict(facecolor="skyblue", color="black"),
    medianprops=dict(color="red", linewidth=2),
    whiskerprops=dict(color="black"),
    capprops=dict(color="black")
)

plt.title("Transaction Amount Distribution")
plt.ylabel("Transaction Amount")
plt.grid(axis='y', linestyle='--', alpha=0.7)
plt.show()
```



Interpretation

The presence of outliers suggests that a few customers make significantly larger purchases.

These customers represent premium buyers and should be targeted with personalized marketing.

9. Key Business Insights

This section summarizes major findings derived from the analysis.

1. A small percentage of customers contribute the majority of revenue.
2. High-frequency customers are the most valuable.
3. Some customers show declining recency and require re-engagement.
4. Major cities dominate the customer base.

In [92]:
```
monthly_customers = transactions.groupby("YearMonth")["CustomerID"].nunique()

retention_rate = monthly_customers.pct_change().mean()

retention_rate
```

Out[92]:
```
np.float64(0.10986147965482872)
```

Repeat Purchase Rate

In [93]:
```
customer_txn_count = transactions.groupby("CustomerID", observed=True).size()
```

In [94]:
```
repeat_customers = (customer_txn_count > 1).sum()
total_customers = customer_txn_count.count()

repeat_purchase_rate = repeat_customers / total_customers
repeat_purchase_rate
```

Out[94]:
```
np.float64(1.0)
```

In [95]:
```
customer_txn_count.value_counts().head()
```

Out[95]:
```
23    96
22    87
24    75
20    71
25    69
Name: count, dtype: int64
```

Avgerage Gap b/w Purchases

In [96]:
```
transactions = transactions.sort_values(["CustomerID", "TransactionDate"])

transactions["PreviousPurchase"] = transactions.groupby("CustomerID", observed=T

transactions["GapDays"] = (transactions["TransactionDate"] - transactions["Previ

average_gap = transactions["GapDays"].mean()

average_gap
```

Out[96]:
```
np.float64(45.529750566893426)
```

Identifying Inactive Customers for 90+ Days

In [97]:
```
reference_date = transactions["TransactionDate"].max() + pd.Timedelta(days=1)
```

In [98]:
```
last_purchase = transactions.groupby("CustomerID", observed=True)["TransactionDa
```

In [99]:
```
inactive_days = (reference_date - last_purchase).dt.days
```

In [100…
```
inactive_customers = inactive_days[inactive_days >= 90]

len(inactive_customers)
```

Out[100…
```
169
```

Ranking Customers by Monetary Value

In [101…
```
df_rfm["MonetaryRank"] = df_rfm["Monetary"].rank(ascending=False)

df_rfm.sort_values("MonetaryRank").head(10)
```

Out[101…

CustomerID	Recency	Frequency	Monetary	R_Score	F_Score	M_Score	RFM_Score
CUST10944	40	31	44784.992188	3	5	5	355
CUST10510	59	30	44367.328125	2	5	5	255
CUST10053	29	30	41674.558594	3	5	5	355
CUST10776	4	33	41050.761719	5	5	5	555
CUST10696	37	30	40035.480469	3	5	5	355
CUST10671	111	34	39763.519531	1	5	5	155
CUST10477	42	36	38015.781250	3	5	5	355
CUST10196	6	35	37738.941406	5	5	5	555
CUST10187	14	30	37627.781250	4	5	5	455
CUST10413	134	36	37479.390625	1	5	5	155

Identifying High-Value Customers in Each City

In [102…

```
merged = pd.merge(df_rfm, customers, on="CustomerID")
high_value_city = merged.sort_values("Monetary", ascending=False)\
                        .groupby("City")\
                        .head(3)

high_value_city[["CustomerID", "City", "Monetary"]]
```

Out[102...

	CustomerID	City	Monetary
944	CUST10944	Ahmedabad	44784.992188
510	CUST10510	Delhi	44367.328125
53	CUST10053	Jaipur	41674.558594
776	CUST10776	Lucknow	41050.761719
696	CUST10696	Delhi	40035.480469
671	CUST10671	Hyderabad	39763.519531
477	CUST10477	Mumbai	38015.781250
196	CUST10196	Chennai	37738.941406
187	CUST10187	Kolkata	37627.781250
413	CUST10413	Delhi	37479.390625
179	CUST10179	Mumbai	36781.371094
831	CUST10831	Jaipur	36714.390625
876	CUST10876	Lucknow	36661.468750
249	CUST10249	Ahmedabad	36299.339844
752	CUST10752	Lucknow	35848.371094
553	CUST10553	Bangalore	35846.250000
312	CUST10312	Ahmedabad	35451.320312
720	CUST10720	Pune	35168.679688
182	CUST10182	Bangalore	34487.058594
470	CUST10470	Bangalore	34133.308594
148	CUST10148	Hyderabad	33617.238281
517	CUST10517	Kolkata	33278.660156
160	CUST10160	Pune	33240.710938
560	CUST10560	Jaipur	33199.468750
649	CUST10649	Mumbai	33119.410156
617	CUST10617	Hyderabad	32881.781250
290	CUST10290	Chennai	32842.082031
854	CUST10854	Kolkata	32764.660156
380	CUST10380	Pune	32543.128906
352	CUST10352	Chennai	32522.820312

Spend Analysis b/w Married & Single Customers

In [103…
```
marital_spending = merged.groupby("MaritalStatus")["Monetary"].mean()
marital_spending
```

Out[103…
```
MaritalStatus
Divorced    23446.660156
Married     23373.568359
Single      22350.005859
Widowed     23063.406250
Name: Monetary, dtype: float32
```

In [104…
```
merged.groupby("MaritalStatus")["Monetary"].sum()
```

Out[104…
```
MaritalStatus
Divorced    5861665.0
Married     5235679.5
Single      5498101.5
Widowed     6457754.0
Name: Monetary, dtype: float32
```

Revenue Growth (Month wise)

In [105…
```
monthly_revenue = transactions.groupby("YearMonth")["TransactionAmount"].sum()

monthly_growth = monthly_revenue.pct_change()

monthly_growth
```

Out[105…

YearMonth	
2020-08	NaN
2020-09	0.643640
2020-10	0.735396
2020-11	0.882887
2020-12	0.699887
2021-01	0.030931
2021-02	0.225372
2021-03	0.293858
2021-04	0.145481
2021-05	-0.008098
2021-06	0.172793
2021-07	0.092356
2021-08	0.112181
2021-09	-0.005814
2021-10	0.238626
2021-11	-0.099577
2021-12	0.179823
2022-01	0.120950
2022-02	-0.041552
2022-03	0.339759
2022-04	0.056910
2022-05	0.002589
2022-06	-0.012695
2022-07	0.072878
2022-08	-0.023981
2022-09	0.099061
2022-10	0.045731
2022-11	0.033883
2022-12	0.073375
2023-01	0.108527
2023-02	-0.081035
2023-03	0.182973
2023-04	0.015333
2023-05	0.077376
2023-06	-0.041337
2023-07	0.061405
2023-08	0.008649
2023-09	0.299647
2023-10	-0.104794
2023-11	0.048308
2023-12	0.077438
2024-01	0.129510
2024-02	-0.103714
2024-03	0.063524
2024-04	0.034183
2024-05	0.206431
2024-06	-0.026412
2024-07	0.076178
2024-08	0.016165
2024-09	-0.072636
2024-10	0.054390
2024-11	-0.073642
2024-12	0.029269
2025-01	0.077295
2025-02	-0.131821
2025-03	0.084486
2025-04	-0.006209
2025-05	0.030245
2025-06	-0.062437

```
2025-07   -0.032291
Freq: M, Name: TransactionAmount, dtype: float32
```

Identifying Peak Shopping Season

In [106…

```
monthly_revenue.sort_values(ascending=False)
```

Out[106…

YearMonth	
2025-01	798789.125000
2024-08	795313.000000
2024-07	782661.250000
2024-10	777659.687500
2025-05	770017.375000
2025-03	752082.062500
2025-04	747412.187500
2024-05	746989.750000
2024-12	741476.437500
2024-09	737544.500000
2024-06	727259.937500
2025-06	721939.750000
2024-11	720391.375000
2025-07	698627.312500
2025-02	693491.875000
2024-01	628087.812500
2024-04	619173.062500
2024-03	598707.375000
2024-02	562946.500000
2023-12	556071.250000
2023-09	549953.750000
2023-11	516104.875000
2023-10	492321.968750
2023-08	423156.187500
2023-07	419527.656250
2023-05	412299.937500
2023-06	395256.812500
2023-04	382688.937500
2023-03	376909.625000
2023-01	346707.656250
2023-02	318612.156250
2022-12	312764.312500
2022-11	291384.000000
2022-10	281834.593750
2022-09	269509.718750
2022-07	251243.125000
2022-08	245218.093750
2022-05	237187.921875
2022-04	236575.468750
2022-06	234176.703125
2022-03	223836.843750
2022-01	174315.593750
2022-02	167072.437500
2021-12	155506.953125
2021-10	146381.546875
2021-11	131805.343750
2021-08	118871.710938
2021-09	118180.617188
2021-07	106881.601562
2021-06	97844.992188
2021-04	84110.148438
2021-05	83429.023438
2021-03	73427.828125
2021-02	56751.058594
2021-01	46313.332031
2020-12	44923.808594
2020-11	26427.519531
2020-10	14035.639648
2020-09	8087.859863

```
2020-08      4920.700195
Freq: M, Name: TransactionAmount, dtype: float32
```

DASHBOARD SUMMARY TABLE

In [107…

```
summary = pd.DataFrame({
    "Total Customers": [transactions["CustomerID"].nunique()],
    "Total Revenue": [transactions["TransactionAmount"].sum()],
    "Avg Transaction": [transactions["TransactionAmount"].mean()],
    "Repeat Purchase Rate": [repeat_purchase_rate],
    "Avg Gap Days": [average_gap]
})

summary
```

Out[107…

	Total Customers	Total Revenue	Avg Transaction	Repeat Purchase Rate	Avg Gap Days
0	1000	23053200.0	1000.138855	1.0	45.529751

Optimization

In [108…

```
optimized_rfm = transactions.groupby("CustomerID", observed=True).agg(
    Monetary=("TransactionAmount", "sum"),
    Frequency=("TransactionAmount", "count"),
    LastPurchase=("TransactionDate", "max")
)

optimized_rfm.head()
```

Out[108…

CustomerID	Monetary	Frequency	LastPurchase
CUST10000	21265.490234	23	2025-07-17
CUST10001	28654.310547	30	2025-06-25
CUST10002	23884.031250	24	2025-07-12
CUST10003	24206.031250	25	2025-05-10
CUST10004	25565.300781	19	2025-07-22

In [109…

```
transactions = transactions.sort_values(
    by=["CustomerID", "TransactionDate"]
)

transactions["GapDays"] = (
    transactions["TransactionDate"] -
    transactions.groupby("CustomerID", observed=True)["TransactionDate"].shift(1
).dt.days
```

In [110…

```
transactions[["CustomerID", "TransactionDate", "GapDays"]].head(15)
```

Out[110…

	CustomerID	TransactionDate	GapDays
19951	CUST10000	2021-05-02	NaN
11952	CUST10000	2021-06-29	58.0
16322	CUST10000	2022-04-14	289.0
5352	CUST10000	2022-04-21	7.0
4308	CUST10000	2022-06-12	52.0
10032	CUST10000	2022-10-01	111.0
761	CUST10000	2022-10-03	2.0
3847	CUST10000	2023-01-31	120.0
14157	CUST10000	2023-02-14	14.0
22088	CUST10000	2023-03-31	45.0
11048	CUST10000	2023-04-15	15.0
16889	CUST10000	2023-05-21	36.0
18033	CUST10000	2023-09-21	123.0
7460	CUST10000	2024-02-04	136.0
11612	CUST10000	2024-02-25	21.0

10. Business Recommendations

1. Focus retention efforts on Champions and Loyal Customers.
2. Launch reactivation campaigns for At Risk customers.
3. Provide loyalty benefits for high Monetary customers.
4. Optimize marketing spend in high-performing cities.
5. Introduce personalized promotions based on RFM score.

11. Conclusion

This project successfully applied RFM segmentation to identify customer value groups.

The analysis demonstrates that targeted retention and personalized marketing strategies can significantly increase revenue and customer lifetime value.

By focusing on high-value segments and re-engaging inactive customers, the business can improve profitability and long-term sustainability.

In [111…
```
df_rfm.to_csv("Final_RFM_Table.csv", index=False)
```

In [112…
```
merged.to_csv("Final_Segmented_Customers.csv", index=False)
```